

Edge deployment: Plant health classification (MobileNet V3) on Raspberry Pi Zero 2 W

Yusuf Savaş

Advisor: Fikret Gürgen

June 3, 2026

TABLE OF CONTENTS

1. Introduction	2
2. Description	3
2.1. Hardware	3
2.2. Software	3
2.3. Use	3
2.4. Diagram	4
3. Requirements	5
3.1. Features	5
4. Designed System	7
4.1. Choices	7
4.2. Pipeline	7
4.3. Quantization	7
5. Implementation	8
5.1. Data preparation	8
5.2. Training pipeline	9
5.3. ONNX export and quantization	10
5.4. On-device runtime (C++)	11
5.5. Web portal	12
5.6. Test bench	13
6. Results	15
6.1. Pi Zero 2 W – FP32 (2-class, 8106 images)	15
6.2. Pi Zero 2 W – FP32 (3-class, 12606 images)	16
6.3. Pi Zero 2 W – INT8/QDQ (3-class, 12606 images)	17
6.4. Dev PC (GPU, reference)	18
6.5. Summary comparison	19
7. Conclusions and Future Work	20
7.1. Future work	20
REFERENCES	22

1. Introduction

Plant pests and diseases destroy up to **40%** of global crop production each year and cost the world economy about **US\$220 billion** annually [1]; major staple crops see **10–40%** yield losses to pathogens [2]. Catching disease early from a leaf image enables timely treatment, less blanket pesticide use, and better yield protection, especially for smallholders who rarely have fast access to a pathologist.

We build an **edge-deployed** classifier on a **Raspberry Pi Zero 2 W** [3]: a camera image is labeled **healthy**, **diseased**, or **background** (non-leaf) using **MobileNet V3** [4] exported to **ONNX** [5] and run with **ONNX Runtime** [6] from **C++** for predictable, low-memory inference. The main focus points are:

- Train on public leaf and background data, then deploy the same preprocessing and graph on the Pi.
- Measure **accuracy and throughput** on device (FP32 baseline vs. **INT8** quantization [7]).
- Expose results through a **web portal** and validate with a **real camera**, test bench, and field-style captures, not only dataset images.

2. Description

2.1. Hardware

Raspberry Pi Zero 2 W was chosen as a **low-cost, resource-constrained** edge node:

Item	Detail
CPU	Quad-core ARM Cortex-A53 (~1 GHz)
RAM	512 MB
Inference	CPU only (no GPU or NPU)

2.2. Software

- **Model:** **MobileNet V3** [4] exported to **ONNX** [5] for deployment.
- **Runtime:** **ONNX Runtime** [6] used from **C++**
- **Pipeline:** **Image in** → **MobileNet-style preprocessing** → **ONNX Runtime inference** → **class scores / label**.
- **Web UI:** a **web portal** on the device (or on the LAN) shows **camera view**, **live predictions**, **summary metrics**, and **throughput (FPS or images per second)**.

2.3. Use

In practice the app does one thing: from a picture of a leaf it guesses whether the plant looks **healthy** or **unhealthy**.

For a **demo**, you open the **web page** on your phone or laptop (on the same Wi-Fi as the device). You show **healthy** and **unhealthy** plants to the camera, or step through example images, and everyone can see the **label** the model picked and **how many frames per second** it is running.

2.4. Diagram

Figure 2.1 shows the logical data flow.

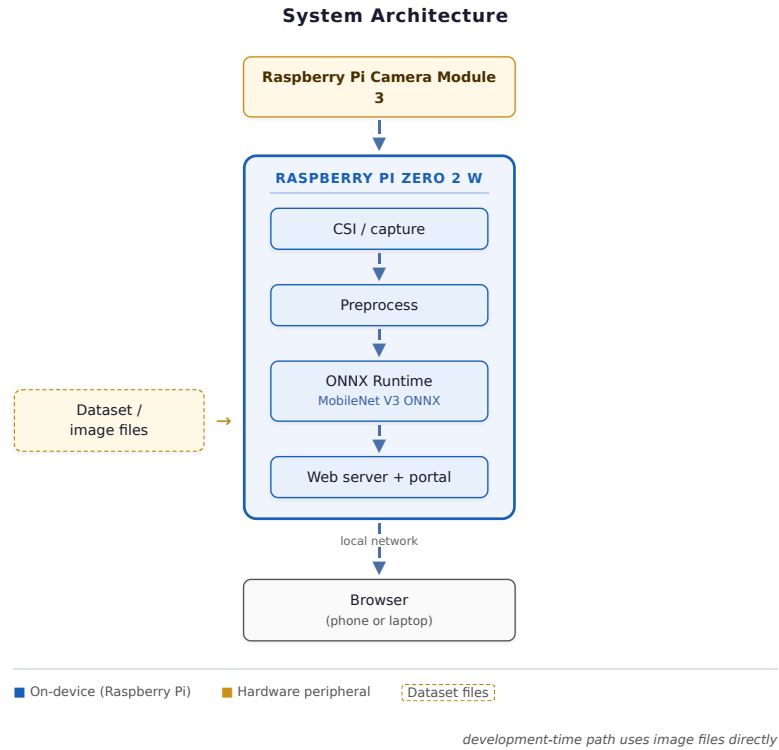


Figure 2.1. System architecture: camera and files into preprocess, ONNX Runtime, web portal, and browser on the LAN.

3. Requirements

3.1. Features

- Open a **web portal** and see **inference results** (predicted class and, where applicable, confidence).
- See **performance at a glance: FPS** (or equivalent throughput) and, if useful, **per-frame or rolling latency**, updated as the system runs.
- Run **single-image** or **stream/batch** inference through the same stack so lab tests and field-style runs share one pipeline.

Layers

Layer	Role
Presentation	Web UI (portal) for status, predictions, FPS, and optional history or session summary.
Application / API	Serves the UI and exposes inference status and metrics to it (WebSocket).
Inference	Loads the MobileNet V3 ONNX graph, applies consistent preprocessing, runs ONNX Runtime on the device CPU.
Acquisition (optional path)	Feeds frames from storage or from a camera into the preprocessor; same graph for file replay and live capture when implemented.
Quantization track	Same topology with INT8 weights for comparison against the FP32 baseline, with accuracy and speed measured under identical UI and metrics.

Offline tests

- Ability to score a **fixed labeled test split** and report **classification metrics** (accuracy, balanced accuracy, precision/recall/F1 for the diseased class, specificity) and a **confusion matrix**, independent of the portal, for rigorous comparison between FP32 and INT8 builds.

4. Designed System

4.1. Choices

- **MobileNet V3** [4] balances accuracy and a small parameter count (**1,519,906** parameters in the trained checkpoint) for edge use.
- **ONNX + ONNX Runtime** [5,6] provides a **deployment-oriented** graph and a runtime tuned for multiple backends; a **native** caller keeps measurements close to how the stack would ship in production.
- **FP32 baseline first**, then **INT8** (or similar) as a controlled experiment: smaller artifacts and faster integer math on CPU, with accuracy tracked against the same test protocol.
- **Web portal** as the default way to **see results and FPS** aligns the system with operator expectations and simplifies demos and field checks.

4.2. Pipeline

- (i) **Input**: image from file, batch folder, or (when available) camera frame.
- (ii) **Preprocess**: resize and normalize to match training/export conventions.
- (iii) **Inference**: ONNX Runtime executes the graph on the CPU.
- (iv) **Output**: class prediction (and optional scores); metrics feed the **portal** and any batch evaluator.
- (v) **Presentation**: portal displays **prediction, FPS / throughput**, and optional aggregates.

4.3. Quantization

A reduced-precision build (e.g. **INT8** weight quantization) targets **smaller footprint and faster inference** on the same CPU, with **accuracy and FPS** compared to the **FP32** baseline under the same preprocessing and portal-facing metrics.

5. Implementation

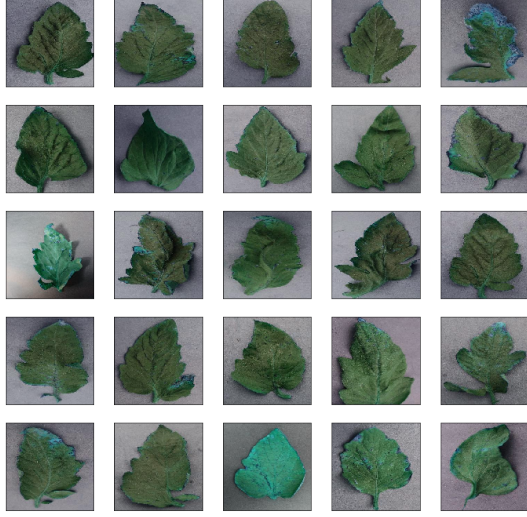
The system is built in two stages that share the same image representation, so results obtained on the workstation carry over to the device. First, a model is trained and exported on a development workstation. Second, an on-device runtime loads that model and serves a web portal on the Raspberry Pi. Both stages feed the network the same input: a 224×224 RGB image, normalized with the standard ImageNet statistics, so the on-device numbers are directly comparable to the reference evaluation.

5.1. Data preparation

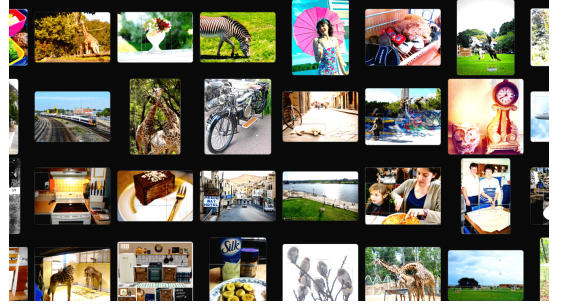
The classifier is trained on a three-class dataset: **healthy**, **diseased**, and **background**. The leaf classes come from **PlantVillage** [8], a large public collection of single-leaf photographs taken on a plain background. PlantVillage originally labels leaves with 38 fine-grained categories (crop $\times$ disease); for this work those are collapsed into a binary health signal: any leaf marked as healthy becomes **healthy**, and every other label becomes **diseased**.

A model trained only on clean leaf photos has no way to say “this is not a leaf,” which is a problem for a live camera that constantly sees hands, tables, and clutter. To handle this, a **background** class is added from **COCO 2017** [9], a dataset of everyday non-leaf scenes, supplemented with a small fraction of synthetic background images. This lets the deployed model reject non-leaf input instead of being forced to guess healthy or diseased.

The combined dataset is split **70 / 15 / 15** into training, validation, and test sets, keeping the three classes separated so that accuracy can be measured on images the model never saw during training.



(a) PlantVillage leaf samples (healthy / diseased)



(b) COCO 2017 background samples

Figure 5.1. Dataset sources. Leaf images provide the healthy and diseased classes; non-leaf scenes provide the background class so the model can reject non-leaf input.

5.2. Training pipeline

Training starts from a **MobileNet V3-small** [4] backbone that has been pre-trained on ImageNet, so it already knows general visual features. Only its final classifier head is replaced with a small dropout-plus-linear layer sized to the three target classes, which lets the network adapt to the leaf task while reusing the pretrained features (transfer learning).

Every image is resized to 224×224 and normalized with the ImageNet mean and standard deviation, matching the pretrained backbone. During training the data is augmented on the fly with horizontal flips, small ($\pm 15^\circ$) rotations, and color jitter, so the model sees slightly varied versions of each leaf and generalizes better. Because the three classes are not equally represented, a weighted sampler is used to draw balanced batches, preventing the model from simply favouring the most common class.

The network is trained for **15 epochs** with a batch size of **32**, using the **Adam**

optimizer and a cross-entropy loss. After each epoch the model is checked against the validation set, and only the best-performing checkpoint (lowest validation loss) is kept. That checkpoint is then scored on the held-out test split to report accuracy, per-class metrics, and a confusion matrix.

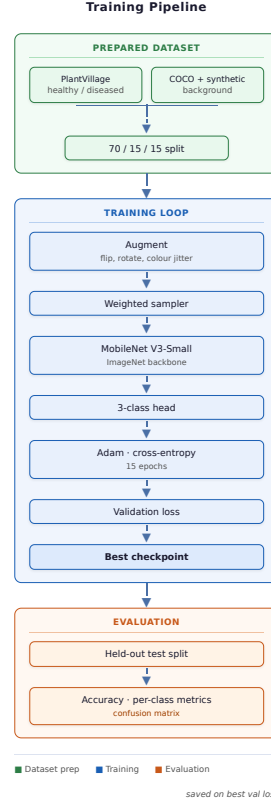


Figure 5.2. Training pipeline: from the prepared dataset through augmentation and balanced sampling into the pretrained MobileNet V3-small, then validation-based checkpoint selection and held-out evaluation.

5.3. ONNX export and quantization

For deployment, the trained model is converted out of its training framework into **ONNX** [5], a portable, framework-independent graph format. The motivation is separation of concerns: training happens in a heavy Python framework, but the device only needs a frozen graph plus a fixed input/output contract (a 224×224 image in,

three class scores out). ONNX captures exactly that, so the same model can be run by a lightweight native runtime on the Pi without dragging the training stack along. The class names are stored inside the exported model so the runtime always knows what each output means, and a parity check confirms the exported graph produces the same outputs as the original model.

A second, reduced-precision build is also produced through **INT8 quantization** [7]. The idea is that weights and activations, normally stored as 32-bit floats, are mapped to 8-bit integers. This shrinks the model roughly four-fold and lets the CPU use cheaper integer arithmetic, which can speed up inference on suitable hardware. The trade-off is precision: representing values with only 256 levels introduces small rounding errors, so quantized models typically lose a little accuracy. This build is kept as a controlled experiment, evaluated against the full-precision baseline on both accuracy and speed under identical conditions.

5.4. On-device runtime (C++)

The edge stack is implemented in **C++** with **ONNX Runtime** [6] so inference stays close to production: no Python interpreter, predictable memory use on 512 MB RAM, and direct access to ARM NEON for preprocessing.

Capture. The Pi Camera Module 3 feeds frames through the **libcamera** [10] stack on Linux. Each frame is converted to an RGB888 buffer suitable for the preprocessor.

Preprocessing. A single fused pass resizes the image to 224×224 , applies optional channel-order correction, and normalizes with the ImageNet mean and standard deviation into the NCHW layout expected by the exported graph. On aarch64 this path uses **NEON** vector instructions to keep preprocessing from dominating latency.

Inference. ONNX Runtime loads the frozen MobileNet V3 graph, binds input

and output tensors, runs one forward pass per frame, then applies softmax and argmax to obtain the predicted class and per-class probabilities.

Serving. A lightweight HTTP server exposes the web portal at the root URL, an MJPEG preview stream for live video, Server-Sent Events for prediction updates, and a metrics endpoint for CPU load, memory, and temperature. A separate batch-evaluation mode replays labeled folders through the same preprocess-and-infer path to produce the accuracy and throughput numbers reported in the Results chapter.

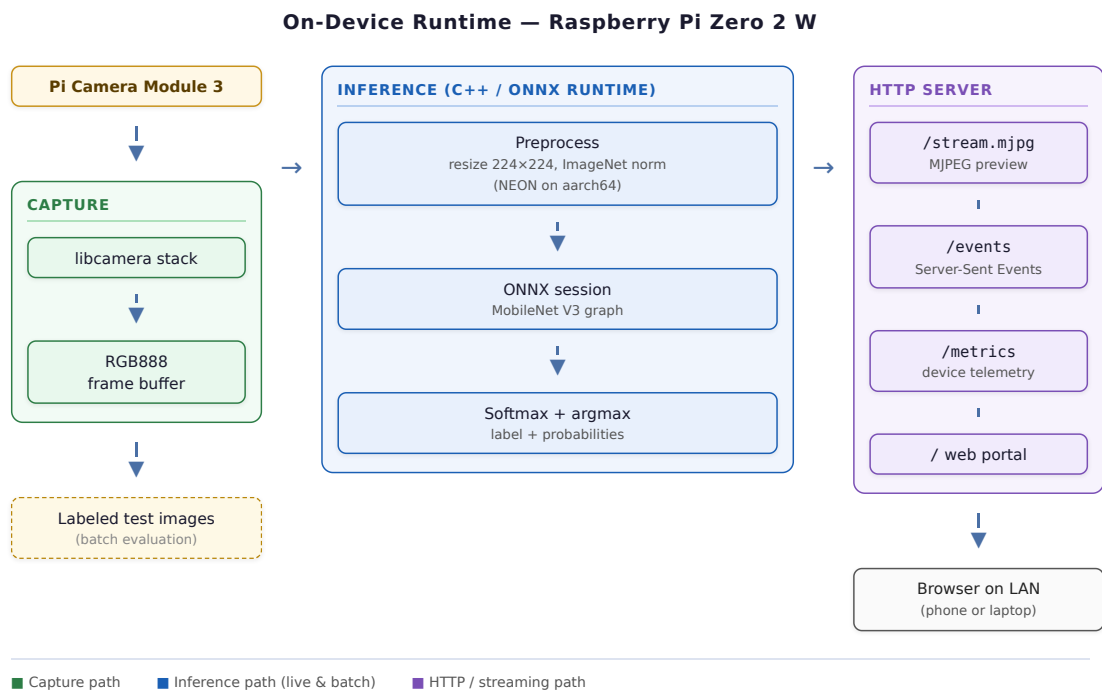


Figure 5.3. On-device runtime: camera capture through preprocessing and ONNX Runtime inference, with HTTP endpoints feeding the browser portal and optional batch evaluation from stored images.

5.5. Web portal

The portal is a self-contained web page served from the device on the local network. It shows the live MJPEG camera preview, the predicted label with per-class probability bars, a rolling throughput estimate (FPS), and device health metrics up-

dated about once per second. Operators can point the camera at healthy or diseased leaves and see predictions change in real time; the background class lets the model reject non-leaf scenes instead of guessing.

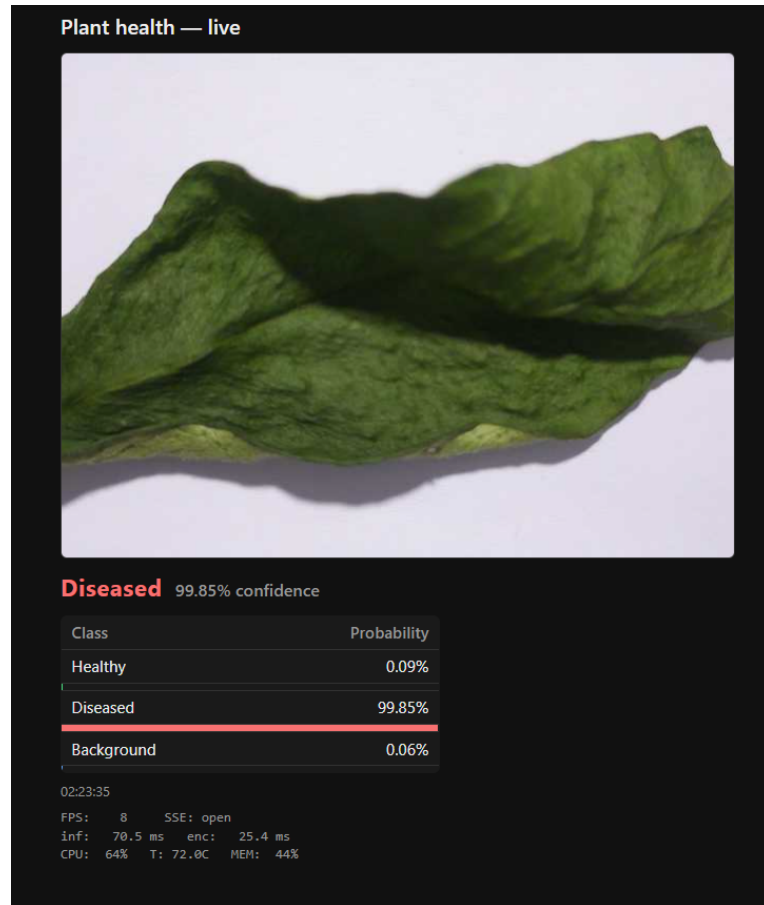
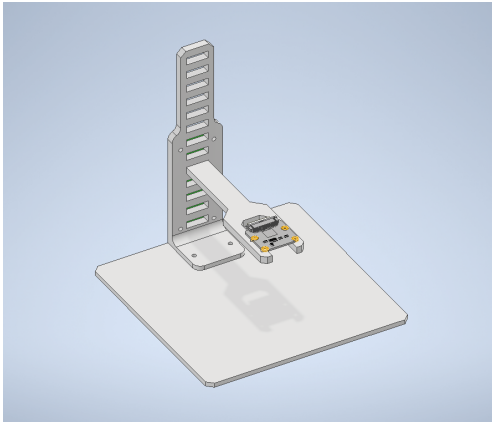


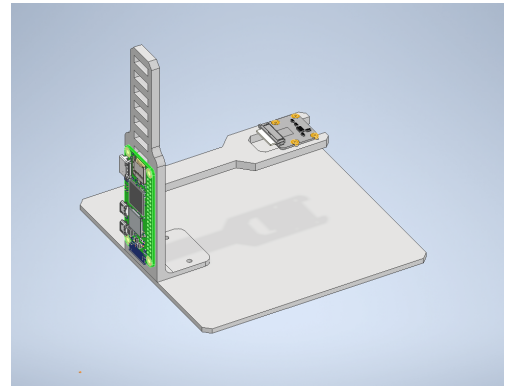
Figure 5.4. Web portal: live preview, predicted label with per-class probabilities, FPS, and device metrics.

5.6. Test bench

Repeatable field-style testing needs a stable camera pose over leaf samples. A **3D-printed test bench** (PLA filament) holds the Pi camera at a fixed height and angle above a flat leaf stage, so successive captures share similar framing and lighting.



(a) CAD front



(b) CAD back

Figure 5.5. 3D-printed (PLA) test-bench camera stand, CAD model.

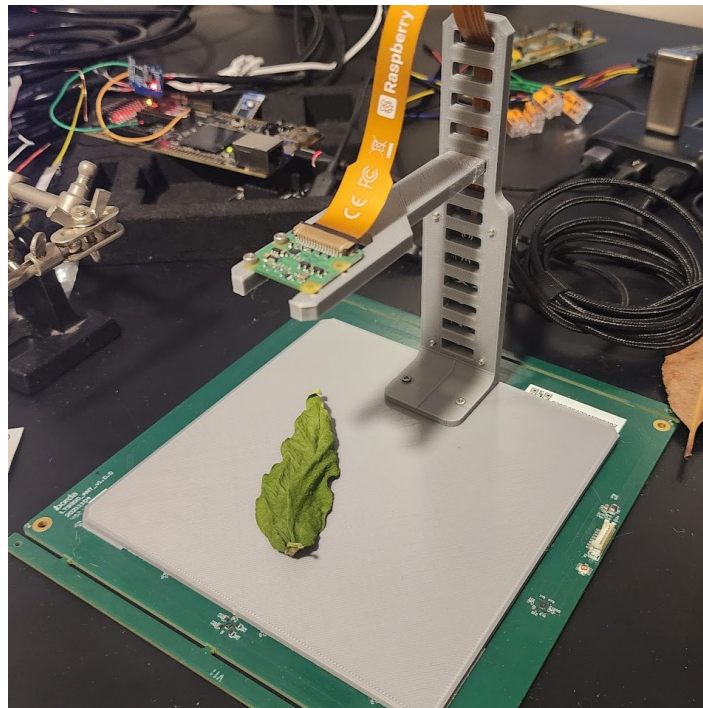


Figure 5.6. Printed test bench in use with the Pi camera and a leaf sample.

6. Results

This chapter reports **benchmark-style** runs on the edge device (Raspberry Pi Zero 2 W, ONNX Runtime via native C++) and a **development workstation** reference (PyTorch on **CUDA**). On-device runs cover both **FP32** ONNX and **INT8/QDQ** ORT builds; sample counts are noted per table (**8106** images for the 2-class leaf-only split, **12606** images for the 3-class split that includes the background class).

6.1. Pi Zero 2 W – FP32 (2-class, 8106 images)

These numbers use **FP32** ONNX on the leaf-only test split (**INT8** not used here).

Metrics (8106 test images)

Metric	Value
Accuracy	0.9983 (99.83%)
Balanced accuracy	0.9978 (99.78%)
Precision (diseased)	0.9988 (99.88%)
Recall (diseased)	0.9988 (99.88%)
F1 (diseased)	0.9988 (99.88%)
Specificity (healthy)	0.9968 (99.69%)

Confusion matrix

	Pred: healthy	Pred: diseased
Actual: healthy	2215	7
Actual: diseased	7	5877

TN: 2215, FP: 7, FN: 7, TP: 5877

Speed (load, preprocess, infer; 8106 images)

Measure	Value
Total time	1009.689 s
Avg / image	124.561 ms
Throughput	8.028 img/s

6.2. Pi Zero 2 W – FP32 (3-class, 12606 images)

The full three-class split adds the **background** class. This run uses **4 threads**.

Overall metrics

Metric	Value
Accuracy	0.9981 (99.81%)
Balanced accuracy	0.9979 (99.79%)
Macro precision	0.9971
Macro recall	0.9979
Macro F1	0.9975

Confusion matrix

	Pred: healthy	Pred: diseased	Pred: background
Actual: healthy	2215	5	2
Actual: diseased	13	5871	0
Actual: background	3	1	4496

Per-class metrics

Speed (load, preprocess, infer; 12606 images, 4 threads)

Class	Precision	Recall	F1	Support
Healthy	0.9928	0.9968	0.9948	2222
Diseased	0.9990	0.9978	0.9984	5884
Background	0.9996	0.9991	0.9993	4500

Measure	Value
Total time	801.375 s
Avg / image	63.571 ms
Throughput	15.730 img/s

6.3. Pi Zero 2 W – INT8/QDQ (3-class, 12606 images)

The INT8/QDQ ORT build trades a few accuracy points for a smaller footprint. Two runs are reported: a single-thread run and a 4-thread run.

Overall metrics

Metric	INT8 (1 thread)	INT8 (4 threads)
Accuracy	0.9721 (97.21%)	0.9705 (97.05%)
Balanced accuracy	0.9691 (96.91%)	0.9667 (96.67%)
Macro precision	0.9605	0.9588
Macro recall	0.9691	0.9667
Macro F1	0.9646	0.9626

Confusion matrix (INT8, 1 thread)

	Pred: healthy	Pred: diseased	Pred: background
Actual: healthy	2106	108	8
Actual: diseased	218	5658	8
Actual: background	3	7	4490

Per-class metrics (INT8, 1 thread)

Class	Precision	Recall	F1	Support
Healthy	0.9050	0.9478	0.9259	2222
Diseased	0.9801	0.9616	0.9707	5884
Background	0.9964	0.9978	0.9971	4500

Speed (load, preprocess, infer; 12606 images)

Measure	INT8 (1 thread)	INT8 (4 threads)
Total time	2085.312 s	1282.092 s
Avg / image	165.422 ms	101.705 ms
Throughput	6.045 img/s	9.832 img/s

6.4. Dev PC (GPU, reference)

Setup (example): Ubuntu 24.04 LTS on **WSL2**; **CPU**: AMD Ryzen 7 5800H;
GPU: NVIDIA GeForce RTX 3050 Laptop (4 GB VRAM).

Test metrics

Metric	Value
Accuracy	0.9980 (99.80%)
Balanced accuracy	0.9972 (99.72%)
Precision (diseased)	0.9983 (99.83%)
Recall (diseased)	0.9990 (99.90%)
F1 (diseased)	0.9986 (99.86%)
Specificity (healthy)	0.9955 (99.55%)
MCC	0.9950
ROC-AUC	1.0000
Parameters	1,519,906

Confusion matrix

	Pred: healthy	Pred: diseased
Actual: healthy	2212	10
Actual: diseased	6	5878

TN: 2212, FP: 10, FN: 6, TP: 5878

Speed (full eval loop)

Measure	Value
Total	10.7903 s
Avg / image	1.331 ms
Throughput	751.23 img/s

6.5. Summary comparison

Build (split, threads)	Accuracy	Throughput
FP32 (2-class, 8106)	99.83%	8.028 img/s
FP32 (3-class, 12606, 4 thr.)	99.81%	15.730 img/s
INT8/QDQ (3-class, 12606, 1 thr.)	97.21%	6.045 img/s
INT8/QDQ (3-class, 12606, 4 thr.)	97.05%	9.832 img/s

On the Pi Zero 2 W, **FP32** keeps accuracy near the reference while **multi-threading** roughly doubles throughput. The **INT8/QDQ** build loses about **2.5–2.7** accuracy points (mostly on the healthy/diseased boundary) and, in this ORT configuration, does not beat multi-threaded FP32 on speed, so FP32 with 4 threads is the strongest on-device operating point measured here.

7. Conclusions and Future Work

On the **Pi Zero 2 W**, **FP32 ONNX** with **ONNX Runtime** stays accurate on the held-out test set ($\sim 99.8\%$) and runs at about **8 images/s** single-threaded, rising to about **16 images/s** with four threads on the three-class split. The **INT8/QDQ** build trades roughly **2.5–2.7** accuracy points for a smaller artifact but, in this ORT configuration, did not outperform multi-threaded FP32 on throughput; multi-threaded FP32 is therefore the strongest on-device operating point measured here.

Why INT8 did not speed up inference. Quantized INT8 graphs are often faster on CPUs that offer wide integer SIMD or dedicated dot-product instructions (for example x86 VNNI or newer ARM cores with int8 matrix extensions). The Pi Zero 2 W uses a **quad-core ARM Cortex-A53** (aarch64): it has NEON for **FP32** float math, but lacks the wider int8 acceleration that many QDQ kernels expect. ONNX Runtime’s INT8 path on this core still pays dequantization and requantization overhead around integer ops, so the theoretical savings from 8-bit arithmetic are largely offset. In practice the NEON-friendly **FP32** kernels remain faster than INT8/QDQ on this hardware.

These results give a solid baseline before further speedups. **Testing with a real camera in real conditions** still matters, because lab-style test images do not cover everything you see in the field.

7.1. Future work

- **Field tests:** systematic runs **outdoors / in greenhouses** with **natural lighting** and varied distances, not only dataset-style images, now that the live camera path exists.
- **Calibration:** optional score calibration or thresholds for diseased vs healthy when moving from lab to field.

- **Speedups:** tune runtime flags and threading for Cortex-A53 to raise images per second.

REFERENCES

1. Food and Agriculture Organization of the United Nations (FAO), “Climate change fans spread of pests and threatens plants and crops, new FAO study”, <https://www.fao.org/newsroom/detail/Climate-change-fans-spread-of-pests-and-threatens-plants-and-crops-new-F> en, 2021, accessed 2026.
2. Savary, S., L. Willocquet, S. J. Pethybridge, P. Esker, N. McRoberts and A. Nelson, “The global burden of pathogens and pests on major food crops”, *Nature Ecology & Evolution*, Vol. 3, No. 3, pp. 430–439, 2019.
3. Raspberry Pi Ltd., “Raspberry Pi Zero 2 W”, <https://www.raspberrypi.com/products/raspberry-pi-zero-2-w/>, 2021, accessed 2026.
4. Howard, A., M. Sandler, G. Chu, L.-C. Chen, B. Chen, M. Tan, W. Wang, Y. Zhu, R. Pang, V. Vasudevan, Q. V. Le and H. Adam, “Searching for MobileNetV3”, *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 1314–1324, 2019.
5. ONNX Community, “ONNX: Open Neural Network Exchange”, <https://onnx.ai>, 2019, accessed 2026.
6. ONNX Runtime developers, “ONNX Runtime: cross-platform, high performance ML inferencing and training accelerator”, <https://onnxruntime.ai>, 2021, accessed 2026.
7. Jacob, B., S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam and D. Kalenichenko, “Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference”, *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2704–2713, 2018.

8. Hughes, D. P. and M. Salathé, “An open access repository of images on plant health to enable the development of mobile disease diagnostics”, *arXiv preprint arXiv:1511.08060*, 2015.
9. Lin, T.-Y., M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár and C. L. Zitnick, “Microsoft COCO: Common Objects in Context”, *European Conference on Computer Vision (ECCV)*, pp. 740–755, Springer, 2014.
10. libcamera developers, “libcamera: An open source camera stack and framework for Linux, Android, and ChromeOS”, <https://libcamera.org>, 2024, accessed 2026.